ROS2

DAY 5 – Manual 1

ROS2 Actions – Fundamentals & Setup

Complete Beginner-Friendly

Objective

By the end of this manual, you will be able to:

1. Understand what **ROS2 Actions** are.
2. Know the difference between **Topics, Services, and Actions**.
3. Create a ROS2 Python package for actions.
4. Define an `.action` file with goal, feedback, and result.
5. Update `package.xml` and `setup.py` to include action interfaces.
6. Build the package and prepare for Python action nodes.

Step 0 – Pre-Requisites

- ROS2 installed (Foxy, Humble, Rolling)
- Python 3 installed
- ROS2 workspace (`ros2_ws`) created
- Basic understanding of nodes, topics, and services (from Days 1–4)

Step 1 – Understand ROS2 Actions

1.1 What is a ROS2 Action?

- Actions allow **asynchronous, long-running tasks**.
- Unlike **Services** (synchronous request/response), actions can provide **feedback while processing**.
- Actions have three parts:

1. **Goal:** request sent by client
2. **Feedback:** intermediate progress sent by server
3. **Result:** final response sent by server

Example:

```
CLIENT → "Move robot 10 meters"
SERVER → feedback: "Moved 2 meters..."
SERVER → feedback: "Moved 5 meters..."
SERVER → result: "Reached 10 meters"
```

1.2 Actions vs Services vs Topics

Feature	Topic	Service	Action
Communication	Continuous stream	Request/Response	Goal/Feedback/Result
Use Case	Sensor data	Commands	Long-running tasks
Response	N/A	Immediate	Asynchronous feedback + result

1.3 Anatomy of an `.action` File

Structure:

```
# Goal (client sends)
<string> command
---
# Result (server returns at end)
<string> result
---
# Feedback (server sends while processing)
<string> feedback
```

- Top section → goal
- Middle section → result
- Bottom section → feedback

Step 2 – Create ROS2 Package for Actions

1. Open terminal:

```
cd ~/ros2_ws/src
```

2. Create Python package `day5_actions`:

```
ros2 pkg create --build-type ament_python day5_actions
cd day5_actions
```

Explanation:

- `--build-type ament_python` → Python ROS2 package

Step 3 – Create `.action` Folder and File

1. Make folder for actions:

```
mkdir -p action
```

2. Create `MoveRobot.action`:

```
nano action/MoveRobot.action
```

3. Paste content:

```
# Goal
float32 distance
---
# Result
string result
---
# Feedback
float32 current_distance
```

Explanation:

- `distance` → goal sent by client
- `result` → server's final response
- `current_distance` → server feedback during execution

Step 4 – Update Package Files

4.1 package.xml

Add dependencies:

```
<build_depend>rosidl_default_generators</build_depend>
<exec_depend>rosidl_default_runtime</exec_depend>
```

4.2 setup.py

Add `.action` file:

```
data_files=[
    ('share/day5_actions/action', ['action/MoveRobot.action']),
],
entry_points={
    'console_scripts': [
        'move_server = day5_actions.move_server:main',
        'move_client = day5_actions.move_client:main',
    ],
},
```

Explanation:

- `data_files` ensures `.action` is installed
- `entry_points` registers server and client nodes

Step 5 – Build the Package

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

Explanation:

- Generates Python action code from `.action` file
- Package ready for server/client implementation

Step 6 – Verify the Setup

1. Check package:

```
ros2 pkg list | grep day5_actions
```

2. Check actions (none yet):

```
ros2 action list
```

- Empty because no nodes running yet
